Title: Determining the time complexity of merge sort and quick sort for 10 different array sizes.

Introduction: Time-complexity is determined to know the taken time from CPU to complete one operation of the program. From the time-complexity we can measure any algorithm's efficiency. Sorting is a very important topic in computer science. To do sorting operation, there are many types of methods. Merge sort and quick sort is one of them. Merge sort is an application of divide and conquer algorithm. It makes some sub-problems of the main problems, solves that then merges them. On the other hand quick sort is kind of Merge sort but it has a great efficiency in worst case than merge sort.

Algorithm: Algorithm for merge sort is given below:

step 1: Start.

Step 2: Define a function Merge (low, mid, high)

Step 3: Declare $n1 = mid - low + 1$ and $n2 = high - mid$ and two array a[n1] and b[n2]

step 4: $i = 0$ for once, and compare $i < n1$ if, yes then go to step 5 else goto step 6.

step 5: a[i] = arr[low+i] then goto step 4 and before that $i = i + 1$.

step 6: $j = 0$ for once and check $j < n2$, if yes then go to step 7 else goto step 8

step 7: b[j] = arr[mid+1+j] then $j = j + 1$ and the goto step 6.

step 8: Initialize, $i = 0$, $j = 0$, $k = low$.

step 9: Check, $i < n1$ and $j < n2$ if yes, check $a[i] <= b[j]$ if yes arr[k] = a[i] and i++ else arr[k] = b[j] and j++ then goto C++ and k++ and if no, goto step 10. else go to step 9.

step 10: Check $i < n1$ if yes arr[k] = a[i], i++, k++ then goto step 10 if no goto step 11.

Step 11 : Check if j< n2 , if yes arr[k] = b[j] and j++ , k++ then goto step 11 if no then goto step 12.

Step 12 : Define a function MergeSort (low, high)

Step 13 : Check low< high , if yes mid =(low+high)/2 and then call MergeSort (low, mid) then MergeSort (mid+1, high) and then Merge(low,mid,high)

Step 14 : End.

Algorithm for quick sort is given below :

Step 1 : Start

Step 2 : Define a function swap (*a, *b) then t = *a , *a = *b and *b = t and goto step 3.

Step 3 : Define a function partition (arr[], low, high) then goto step 4.

Step 4 : Declare pivot = arr[high] , i = low - 1 gotostep5.

Step 5 : Initialize for once j = low then check j≤high - 1 and if yes then check arr[j] ≤ pivot if yes i++ and swap (&arr[i], &arr[j]) then goto step 5. if no goto step 6,

Step 6 : swap (&arr[i+1], &arr[high]) and return (i+1)

step 7 : Define a function QuickSort(arr[], low, high) then check low < high, if yes goto step 8. else goto step 9.

step 8 : pi = partition(arr[], low, high) then call QuickSort(arr, low, pi-1) then QuickSort(arr, pi+1, high) then goto step 9.

step 9 : End.

Code : Merge Sort

```
#include<stdio.h>
#include<bits/stdc++.h>
using namespace std;
#define ll long long
#define inf 1e18
ll arr[10000], c=0;

void Merge(ll low, ll mid, ll high){
ll i,j,k;
c+=5;
ll n1= mid-low+1;
ll n2= high-mid;
ll a[n1], b[n2];
```

```
for(ll i=0; i<n1; i++){
  c+=4; a[i]=arr[low+i]; }
for(ll j=0; j<n2; j++) {
  c+=5;
  b[j] = arr[mid+1+j];
}
i=0; j=0; k= low;
while( i<n1 && j<n2){
   c+=5;
   if(a[i]<=b[j]){
    c+=2;
    arr[k]= a[i];
    i++; }
   else { c+=2; arr[k]=b[j]; j++; }
   c++; k++; }
 while( i<n1) {
   c+=4; arr[k]=a[i]; i++; k++;}
 while (j<n2) {
   c+=4; arr[k]=b[j]; j++; k++;}
}

void MergeSort (ll low, ll high)
{  c+=7;
   if ( low<high) {
   ll mid = (low+high)/2;
   MergeSort (low, mid);
   MergeSort (mid+1, high);
   Merge( low, mid, high); }}
```

```
int main()
{
ll n;
cout<<"Time Complexity of merge sort for different 10
input sizes\n"<<endl;
for(ll i=0;i<10;i++){
   cout<<"Enter the size of array: ";
  cin>>n;
  for(ll i=0; i<n; i++)
     arr[i]= rand()%10000000000-1000;
  c++;
  MergeSort(0,n-1);
  cout<<"Time Complexity for input size "<<n<<" is: "
  <<(double) c/320000000000<<" seconds\n"<<endl;
  c=0;
 }
 return 0;
}
```

Quick Sort

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
#define inf 1e18
ll arr[10000], c=0;

void swap( ll* a, ll* b)
{
    c+=3;
    ll t = *a;
     *a = *b
     *b = t;
}

ll partition (ll arr[], ll low, ll high)
{
  c+=3;
  ll pivot = arr[high];
  ll i = low-1;
  c++;
  for(ll j=low; j<=high-1 ; j++)
  {
     c+=4;
     if(arr[j]<=pivot)
     { c+=2; i++; swap(&arr[i], &arr[j]); }
     }
    c+=3; swap(&arr[i+1],&arr[high]); return(i+1);}
```

```
void QuickSort (ll arr[], ll low, ll high)
{  c++;
   if (low< high)
   {
     c+=4;
     ll  pi= partition(arr, low, high);
     QuickSort (arr, low, pi-1);
     QuickSort (arr, pi+1, high);
   }
}

int main()
{
  ll n;
  cout<<"Time Complexity of Quick sort for different 10
  input sizes \n"<<endl;
  for(ll i=0; i<10; i++) {
    cout<< "Enter the size of array: ";
    cin>>n;
    for(ll i=0; i<n; i++)
      arr[i]= rand()%10000000000-1000;
    c++;
    QuickSort(arr,0,n-1);
    Cout<< "Time complexity for input size "<<n<<" is: "<<
    (double) c/3200000000000 << " seconds "<<endl;
    c=0; }
    return 0;
}
```

Sample Input/Output :

Time Complexity of Merge Sort for Diffenent 10 input sizes

Enter the size of array: 1000
Time complexity for input size 1000 is: 4.33091e-007 seconds.

Enter the size of array : 2000
Time complexity for input size 2000 is: 9.44303e-007 seconds

Enter the size of array: 3000
Time complexity for input size 3000 is: 1.48934e-006 seconds

Enter the size of array: 4000
Time complexity for input size 4000 is: 2.04453e-006 seconds

Enter the size of array : 5000
Time complexity for input size 5000 is: 2.62528e-006 seconds

Enter the size of array: 6000
Time complexity for input size 6000 is: 3.21335e-006 seconds

Enter the size of array: 7000
Time complexity for input size 7000 is: 3.80544e-006 seconds

Enter the size of array : 8000
Time complexity for input size 8000 is: 4.40232e-006 seconds
Enter the size of array : 9000
Time complexity for input size 9000 is: 5.01748e-006 seconds

Enter the size of array : 10000
Time complexity for input size 10000 is: 5.64198e-006 seconds

Time Complexity of Quick Sort for different 10 input sizes :

Enter the size of array : 1000
Time complexity for input size 1000 is: 2.44681e-007 seconds

Enter the size of array: 2000
Time complexity for input size 2000 is: 6.32122e-007 seconds

Enter the size of array : 3000
Time complexity for input size 3000 is: 9.09959e-007 seconds

Enter the size of array : 4000
Time complexity for input size 4000 is: 1.15903e-006 seconds

Enter the size of array : 5000
Time complexity for input size 5000 is: 1.5972e-006 seconds

Enter the size of array : 6000
Time complexity for input size 6000 is: 2.35923e-006 seconds

Enter the size of array : 7000
Time complexity for input size 7000 is: 2.16753e-006 seconds

Enter the size of array : 8000
Time complexity for input size 8000 is: 2.82538-006 seconds

Enter the size of array : 9000
Time complexity for input size 9000 is : 2.99998e-006 seconds

Enter the size of array : 10000
Time complexity for input size 10000 is : 3.70864e-006 seconds

Discussion & Analysis :

In this experiment the time-complexity for 10 different large input size on merge sort and quick sort were calculated. From the above result in output it is seen that there is some difference in time-complexity between merge sort and quick sort for large amount of input size.